

Reviewer Pack — Continued Fraction Depth of Spectral Ratio Bounds Discrepanc...

Entry 71ce01b16c1e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 00:31:39 UTC

Continued Fraction Depth of Spectral Ratio Bounds Discrepancy

Entry ID: 71ce01b16c1e **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 00:31:39 UTC

1. Conjecture

Field A (mathematical branch): Diophantine approximation (continued-fraction expansions and partial-quotient growth, in the spirit of Khinchin and Gauss-Kuzmin metric theory) **Field B** (complexity object): Discrepancy-based randomized two-party communication complexity lower bounds for ± 1 sign matrices

Statement:

Let $M \in \{-1, +1\}^{N \times N}$ with $N = 2^n$ have singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_N$, and let $\rho(M) = \sigma_2 / \sigma_1 \in [0, 1]$. Let $(a_1, a_2, \dots, a_K)$ be the simple continued-fraction expansion of $\rho(M)$ truncated at $K = \lceil \log_2 N \rceil$ partial quotients (computed via the Euclidean algorithm on a $4N$ -digit decimal truncation of ρ), and define $S(M) = \sum_{i=1}^K \log(1 + a_i)$. Then for every $n \geq 3$ and every ± 1 sign matrix M of size $2^{n \times 2}n$, $\text{disc}(M) \cdot N \geq c \cdot \sigma_1(M) / (1 + S(M))$ for an absolute constant $c = 0.05$, where $\text{disc}(M)$ is the combinatorial rectangle discrepancy of M .

Rationale (proposer's reasoning):

Highly structured matrices (Hadamard, Sylvester, AND/XOR-lifted indicators) have spectra that are integer or low-height rational, forcing σ_2 / σ_1 to be exactly 0, 1, or a fraction with tiny partial quotients — yielding small $S(M)$ and hence the well-known large $\text{disc} \cdot N \approx \sigma_1$. Generic matrices satisfy Gauss-Kuzmin statistics with partial quotients distributed like $1/(k(k+1))$ and $S(M) = \Theta(\log N)$, exactly compensating their $\sigma_1 = \Theta(\sqrt{N})$ into $\text{disc} \cdot N = \Theta(\sqrt{N})$. The Diophantine-approximation lens has, to my knowledge, never been applied to the spectral side of the discrepancy method, even though Khinchin-style metric theory is the natural quantitative refinement of ‘how rational is the spectral ratio’.

Taxonomy category: COMM_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d201a96275cda6b3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across all 140 (ensemble, n, seed) triples with $n \in \{3..9\}$, 4 ensembles, 5 seeds, compute indicator $I = [\text{disc}(M) \cdot N \geq 0.05 \cdot \sigma_1(M) / (1 + S(M))]$. Conjecture is SUPPORTED iff $\text{mean}(I) > 0.90$ AND no Hadamard instance (ensemble ii) violates the bound; FALSIFIED otherwise.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.78	SAFE	SAFE
ALGEBRIZATION	SAFE	0.84	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - discrepancy sign matrix singular values communication complexity lower bound - continued fraction partial quotients spectral ratio Khinchin Diophantine matrix-rectangle discrepancy second singular value ± 1 matrix randomized communication

Top relevant hits considered: - [http://arxiv.org/abs/1210.1461v1] A Scalable CUR Matrix Decomposition Algorithm: Lower Time Complexity and Tighter Bound - [http://arxiv.org/abs/cs/0111062v2] One-way communication complexity and the Neciporuk lower bound on formula size - [http://arxiv.org/abs/cs/9908001v1] A Lower Bound on the Average-Case Complexity of Shellsort - [http://arxiv.org/abs/2209.08318v3] Hausdorff dimension of sets with restricted, slowly growing partial quotients in semi-regular continued fractions - [http://arxiv.org/abs/1510.00068v2] On the complete solution of the general quintic using Rogers-Ramanujan continued fraction - [http://arxiv.org/abs/1111.6023v2] On a General Sextic Equation Solved by the Rogers Ramanujan Continued Fraction - [http://arxiv.org/abs/2207.14810v3] Simplifying a classical-quantum algorithm interpolation with quantum singular value transformations - [http://arxiv.org/abs/2004.09833v2] Expressing the largest eigenvalue of a singular beta F-matrix with heterogeneous hypergeometric functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i+1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
```

```

    A[i], A[max_row] = A[max_row], A[i]
    factor = A[i][i]
    for j in range(i, n):
        A[i][j] /= factor
    for k in range(n):
        if k != i:
            factor = A[k][i]
            for j in range(i, n):
                A[k][j] -= factor * A[i][j]
    return [row[n-1] for row in A]

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    sign = 1
    for i in range(n):
        minor = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(minor)
        sign *= -1
    return det

def svd(A):
    U, S, Vt = [], [], []
    n = len(A)
    for i in range(n):
        U.append([random.choice([-1, 1]) for _ in range(n)])
        S.append(random.uniform(0.5, 2))
        Vt.append([random.choice([-1, 1]) for _ in range(n)])
    return U, S, Vt

def continued_fraction(x, K):
    a = []
    for _ in range(K):
        a.append(int(x))
        x = 1 / (x - int(x))
    return a

def log_sum(a):
    s = 0
    for ai in a:
        s += math.log(1 + ai)
    return s

def comb_discrepancy(M):
    n = len(M)

```

```

max_disc = 0
for _ in range(400):
    A, B = set(), set()
    while len(A) < n or len(B) < n:
        i, j = random.randint(0, n-1), random.randint(0, n-1)
        if i not in A and j not in B:
            A.add(i)
            B.add(j)
    disc = abs(sum(M[i][j] for i in A for j in B)) / (n**2)
    max_disc = max(max_disc, disc)
return max_disc

def generate_matrix(n, M_type):
    if M_type == "uniform":
        return [[random.choice([-1, 1]) for _ in range(2**n)] for _ in range(2**n)]
    elif M_type == "sylvester_hadamard":
        H = [[0]*2**n for _ in range(2**n)]
        H[0][0] = 1
        for i in range(1, 2**n):
            H[i][:i] = [H[i-1][j] ^ H[i-1][j^i] for j in range(i)]
            H[i][i:] = [H[i-1][j] ^ H[i-1][j^i] for j in range(i, 2**n)]
        return H
    elif M_type == "and_xor":
        n_bits = int(math.log2(n))
        return [[int(x[i] and y[i]) ^ int(x[i] or y[i]) for i in range(n_bits)] for x in iter...]
    elif M_type == "disjointness":
        n_bits = int(math.log2(n))
        return [[int(x[i] and y[j]) for i in range(n_bits) for j in range(n_bits)] for x in iter...]
    else:
        return None

def spectral_ratio(M):
    U, S, Vt = svd(M)
    sigma_1 = S[0]
    sigma_2 = S[1]
    rho = sigma_2 / sigma_1
    a = continued_fraction(rho, int(math.log2(2**n)))
    return sigma_1, sigma_2, a

n_values = [3, 4, 5, 6, 7, 8, 9]
ensemble_types = ["uniform", "sylvester_hadamard", "and_xor", "disjointness"]
results = []

for n in n_values:
    for M_type in ensemble_types:
        for _ in range(5):
            M = generate_matrix(n, M_type)
            sigma_1, sigma_2, a = spectral_ratio(M)
            S = log_sum(a)
            disc = comb_discrepancy(M)
            I = disc * 2**n >= 0.05 * sigma_1 / (1 + S)
            results.append({
                "metric_name": "I",
                "metric_value": int(I),
                "instances_tested": 1,
                "conjecture_holds": I,
            })

```

```

        "counterexample": "" if I else f"{M_type} matrix of size {2**n}x{2**n}"
    })

mean_I = sum(result["metric_value"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

return {
    "metric_name": "I",
    "metric_value": mean_I,
    "instances_tested": len(results),
    "conjecture_holds": mean_I > 0.9 and all(result["counterexample"] == "" for result in results),
    "counterexample": next((result["counterexample"] for result in results if not result["conjecture_holds"]), "")
}

if __name__ == "__main__":
    seeds = [int(seed) for seed in sys.argv[1:]] or [11, 23, 37, 53, 71]
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_I = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["counterexample"] == "" for result in results if result["conjecture_holds"]):
        print(f"RESULT: SUPPORTED mean={mean_I} std=0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] and result["counterexample"] != "" for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{next(result['counterexample'] for result in results if not result['conjecture_holds'])}\"")
    else:
        print("RESULT: INCONCLUSIVE support_fraction too low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5248d06.py", line 137, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5248d06.py", line 113, in run_trial
    M = M_type()
        ~~~~~^
TypeError: run_trial.<locals>.<lambda>() missing 1 required positional argument: '_'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `TypeError` (lambda missing positional argument `'_'`) before any trials produced data, so neither the support nor falsification criteria can be evaluated. | next: Fix the `M_type` lambda signature in `run_trial` so all four ensembles can be instantiated uniformly, then rerun the 140-triple sweep.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	107251
2	preregistration	claude_max	opus	0	0	6099
3	novelty	claude_max	opus	0	0	3701
4	novelty	claude_max	opus	0	0	6904
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18014
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19430
7	judge	claude_max	opus	0	0	3862

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 165262 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/71ce01b16c1e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/71ce01b16c1e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/71ce01b16c1e.tar.gz` (if generated)